Project Definition Document

Campus Data Orchestrator — An MCP-Based Campus Data Orchestrator

Course	BIL 481 Software Engineering, Fall 2026, TOBB University of Economics and Technology
Assignment	Project Assignment 1
Group	Group-ASYM
Group members	Arda Çelik (241101065), Murat Can Işık (241101013), Yunus Emre Biçer (241101009), Ahmet Sefa Efendioğlu (241101066)
Contributors to this document	**Arda Çelik** (author, all content sections), **Ahmet Sefa Efendioğlu** (cross-review, table of contents, PDF conversion)
Version / Date	1.0 (final), 1 November 2026
Related documents	Concept Notes (`docs/Concept_Notes.pdf`), Requirements Document, Project Plan Document

Table of Contents

1. Project Name
2. Project Summary
3. Objectives
4. Scope
   - 4.1 In Scope
   - 4.2 Out of Scope
   - 4.3 External Dependencies and Access Status
5. Target Audience
6. Key Features
7. Deliverables
8. Budget and Resources
9. Risks and Mitigation Strategies
10. Project Success Criteria
11. Changes Since the Concept Notes
12. Document-Specific Task Matrix
13. Revision History

1. Project Name

Campus Data Orchestrator (full title: *Model Context Protocol (MCP) Based Campus Data Orchestrator*).

The name describes what the system does: it *orchestrates* data from several separate campus services and exposes it through one open standard, the Model Context Protocol (MCP), so that both a web client and AI assistants can use it.

2. Project Summary

Students and staff at a university campus depend on several unconnected digital systems for everyday decisions: the Student Information System (SIS) for class schedules, the dining portal for the daily cafeteria menu, the library portal for seat occupancy, and the transport portal for campus shuttle ("ring") times. Each system has its own web page and data format, so answering a simple question such as "*What is for lunch today, and can I catch a ring after my 10:30 class?*" requires opening three or four different pages and combining the information by hand.

Campus Data Orchestrator solves this fragmentation by placing an MCP server between campus data sources and their users. Each campus service is wrapped by a connector and exposed as a standardized MCP *tool* with a strict JSON Schema. A web client offers two ways to use these tools: a **Campus at a Glance dashboard** that shows frequently needed information (menu, shuttles, library occupancy) instantly without calling an AI model, and a **conversational assistant** that uses a Large Language Model (LLM) to combine several tools and answer multi-service questions in one reply. The system is designed to stay useful when a campus service is down (cached data with a clear "stale" notice), to keep student credentials away from the LLM, and to run fully on synthetic (mock) data when real campus interfaces are not accessible.

3. Objectives

By the end of the project (15 January 2027) the team aims to achieve the following product objectives:

ID	Objective	Linked features
O-1	Expose four campus services (cafeteria menu, shuttle schedule, library occupancy, personal course schedule) as MCP tools that any MCP-compatible client can discover and call.	F-1
O-2	Provide a web dashboard that shows the four most frequently needed pieces of campus information in a single screen without LLM cost or delay.	F-2, F-6
O-3	Provide a conversational assistant that answers compound, multi-service questions in a single turn, in Turkish or English.	F-3, F-6
O-4	Keep the service usable during upstream outages through caching, stale-if-error fallback, and circuit breaking, and tell the user clearly when data is not fresh.	F-4
O-5	Protect student identity: authenticate at an API gateway, never pass tokens to the LLM, and remove personally identifiable information (PII) from prompts.	F-5
O-6	Make the whole system runnable and testable without access to real campus systems, using connectors with synthetic data generators.	F-7

4. Scope

The scope is deliberately limited to a **small, complete version** that four members can implement, test, and demonstrate within the planned 70 person-hours (see the Project Plan Document). Essential functions are separated from optional extensions; the priority of each requirement is given in the Requirements Document.

4.1 In Scope

- An MCP server (TypeScript/Node.js) registering four tools: `get_cafeteria_menu`, `get_shuttle_schedule`, `get_library_occupancy`, `get_course_schedule`, available over Streamable HTTP (and stdio for local MCP clients).
- One connector per service behind a common connector interface, each with a **mock mode** that returns realistic synthetic data. Live mode is implemented only for sources whose access is confirmed (see 4.3).
- Caching in a single Redis instance with per-tool TTLs, stale-if-error fallback, and a circuit breaker per connector.
- An API gateway (Fastify) that verifies JWTs issued by a **local mock SSO issuer**, applies rate limiting, removes the token, and injects the verified student ID into an internal header.
- An AI agent component that uses an LLM API (Gemini by default, behind a provider adapter) to plan tool calls and write the combined answer.
- A Next.js web client with two views: the conversational assistant (with streamed answers and tool-call badges) and the Campus at a Glance dashboard, plus a rule-based Turkish/English intent router that sends simple queries straight to the dashboard data path.
- Clear user-facing handling of errors, missing information, slow responses, and interrupted connections.
- PostgreSQL (via Prisma) for tool-call telemetry and audit logs.
- Local deployment with Docker Compose, automated tests (unit, integration, load), and project documentation.

4.2 Out of Scope

- Real integration with TOBB ETÜ's production SIS, SSO, shuttle GPS, or library turnstile systems. These are replaced by mock connectors unless official access is granted.
- Production hosting, high availability, Redis Cluster mode, and Kubernetes. The system runs locally (optionally a free-tier demo host); multi-instance scaling is only demonstrated with two local containers.
- Mutual TLS between all internal services (optional extension; TLS is used on the public endpoint only).
- Native mobile applications, push notifications, and offline mode.
- Write operations on campus systems (e.g., booking a library seat, registering for courses, paying for meals).
- Training or fine-tuning an LLM, and voice input/output.
- Personalization beyond the student's own course schedule (no recommendation engine).

4.3 External Dependencies and Access Status

Following the TA's guidance to verify external dependencies early, each dependency is labeled as **confirmed** or **planned**. Anything not confirmed is implemented with representative mock data, and the mock is used in all automated tests regardless of status.

Dependency	Used by	Access status (27 Oct 2026)	Approach
LLM API (Google Gemini API key from Google AI Studio)	AI agent (F-3)	**Confirmed**: team members can create API keys; free tier is rate-limited	Use free tier during development; provider adapter allows switching provider
TOBB ETÜ cafeteria menu web page	Cafeteria connector	**Planned**: public page, scraping terms and page structure to be checked by 2 Nov 2026	Mock connector first; live scraper only if access is confirmed

Dependency	Used by	Access status (27 Oct 2026)	Approach
Campus shuttle (ring) timetable	Shuttle connector	**Planned**: no public API known; a published timetable may be transcribed	Mock connector with timetable-based synthetic departures
Library occupancy feed	Library connector	**Unavailable**: no public feed	Mock connector only (`isSimulated: true` in responses)
Student Information System (course schedule)	Schedule connector	**Unavailable**: no student API access	Mock connector with synthetic student schedules
University SSO (OAuth2/OIDC)	API gateway	**Unavailable**	Local mock SSO issuer that signs test JWTs with the same claims structure
Redis, PostgreSQL, Node.js libraries	All back-end components	**Confirmed**: open source, run locally via Docker	Docker Compose

Where mock data is used, the user interface and the API responses mark it as simulated so that demonstrations never present synthetic data as real campus data.

5. Target Audience

Audience	Needs	How the project serves them
Undergraduate and graduate students (primary users)	Quick answers about meals, shuttles, study space, and their own classes, often on a phone between classes	Dashboard for one-glance answers; assistant for combined questions in Turkish or English
Faculty and staff	Shuttle times, cafeteria menu, library availability	Dashboard and assistant (without personal schedule data)
Campus IT and developers	A standard way to expose campus data to AI tools and applications	MCP server with documented tool schemas and a pluggable connector interface
Course staff (instructor, TA)	Evaluation of a well-engineered, demonstrable system	Mock mode makes every feature demonstrable without campus credentials

6. Key Features

ID	Feature	Description
F-1	**Campus MCP Tool Server**	Exposes four campus services as MCP tools with strict JSON Schemas; any MCP client (e.g., MCP Inspector or a desktop AI assistant) can list and call them.
F-2	**Campus at a Glance Dashboard**	Four widget cards (cafeteria menu, next shuttles, library occupancy, service status) with manual refresh and "last updated" times, served from cache without LLM calls.
F-3	**Conversational Campus Assistant**	Chat interface where an LLM agent plans multiple tool calls and returns one combined, streamed answer, with badges showing each tool call's status and duration.
F-4	**Resilient Data Access**	Per-tool caching, stale-if-error fallback with an `isStale` flag and timestamp, and a circuit breaker that isolates a failing service.

ID	Feature	Description
F-5	**Secure Identity Handling**	API gateway verifies the student's token, strips it, and passes only a verified student ID; PII is removed before any prompt reaches the LLM; per-student rate limiting.
F-6	**Bilingual Intent Router**	Client-side rule-based router that recognizes simple Turkish and English queries (e.g., "yemekte ne var", "next ring") and answers them directly; compound or ambiguous queries go to the assistant.
F-7	**Pluggable Connectors with Mock Mode**	Common connector interface; each connector can switch between live and mock mode through configuration, without code changes.

7. Deliverables

#	Deliverable	Format / location	Planned date
D-1	Concept Notes (Assignment 0)	PDF in `docs/` and Google Drive	Completed (October 2026)
D-2	Project Definition, Requirements, and Project Plan documents (Assignment 1)	Markdown + PDF in `docs/PA-1/`; PDFs in Drive folder `PA-1-01-11-2026`	1 Nov 2026
D-3	Design documentation (architecture, interface and data design) and repository skeleton	Markdown/PDF in `docs/`, code skeleton in repository	20 Nov 2026 (exact Assignment 2 date to be announced by course staff)
D-4	Working software: MCP server, gateway, agent, connectors, web client	Source code in GitHub repository, `docker-compose.yml`	25 Dec 2026
D-5	Test suite and test report (unit, integration, load, usability results)	Code in repository, report in `docs/`	15 Jan 2027
D-6	Final project documentation, user/installation guide, and demo	PDF in `docs/`, README, live demo	15 Jan 2027

8. Budget and Resources

Human resources. Four team members, each planning 17.5 hours over the term (70 person-hours in total). Roles and the effort breakdown are given in the Project Plan Document.

Software tools (all free or open source).

- Languages and frameworks: TypeScript, Node.js, Fastify, `@modelcontextprotocol/sdk`, Next.js, React, Tailwind CSS.
- Data: Redis (single instance), PostgreSQL with Prisma ORM.
- Testing: Jest, Supertest, MCP Inspector, a load-testing tool (k6 or autocannon).
- Collaboration: GitHub (repository, Issues, Projects board, pull requests), Google Drive, Google Meet, WhatsApp.
- Development: VS Code, Docker Desktop.

Hardware. Members' own laptops. No servers are purchased; all services run locally in Docker containers.

Learning resources. MCP specification and SDK documentation, Fastify and Next.js documentation, course lecture notes, TA office hours (Thursday and Friday, 16:30–18:20, TM-213).

Time. 19 October 2026 to 15 January 2027 (about 13 weeks), divided into four phases (see Project Plan Document).

Budget considerations.

Cost item	Planned approach	Expected cost
LLM API usage (tokens)	Free tier for development; intent router keeps simple queries away from the LLM; short system prompts; usage monitored weekly	0 TL planned
Server infrastructure	Local Docker Compose; optional free-tier host for the demo	0 TL
Software licenses	Open-source tools only	0 TL
Contingency	If the free LLM tier is not sufficient for testing and the demo, the team may enable a paid key with a spending cap agreed in advance by all members	Small, capped; only with team approval

9. Risks and Mitigation Strategies

Likelihood (L) and impact (I) are rated Low / Medium / High. The same risk IDs are used in the Project Plan Document.

ID	Risk	L	I	Mitigation strategy
R-1	No access to real campus data sources (SIS, library, shuttle, SSO)	High	Medium	Mock connectors and mock SSO from the start (F-7); live mode only for confirmed sources; all tests run in mock mode
R-2	External service outage or slow response during use or demo	Medium	High	Caching with stale-if-error, 800 ms upstream timeout, circuit breaker; demo runs in mock mode
R-3	LLM API cost, quota limits, or rate limiting	Medium	Medium	Free tier, intent router bypass for simple queries, response caching for tools, provider adapter to switch LLM provider
R-4	LLM gives wrong or invented answers (hallucination)	Medium	High	Agent answers only from tool results; low temperature; tool results shown as badges; scripted compound-query tests
R-5	Leakage of student tokens or PII to the LLM provider	Low	High	Token termination at the gateway; PII scrubbing; automated test that checks logged prompts contain no IDs, names, or e-mails
R-6	Scope too large for 70 person-hours	High	High	Must/Should/Could priorities; optional items (mTLS, Redis Cluster, live scrapers) are dropped first; scope reviewed at the end of each phase
R-7	Team member unavailable (illness, exams, other course load)	Medium	Medium	Every module has a secondary reviewer who knows it through code review; tasks tracked on the GitHub Projects board
R-8	Unfamiliar technologies (MCP SDK, SSE, Redis) slow development	Medium	Medium	Early technical spike in the Design phase; small vertical prototype (one tool end-to-end) before full development
R-9	Integration problems between separately developed components	Medium	Medium	Interfaces and JSON Schemas fixed in the Design phase; small, frequent pull requests into `main`; integration tests in CI on every pull request
R-10	Merge conflicts or lost work in GitHub	Low	Medium	Feature branches, pull requests into `main` with review, no direct commits to `main`

10. Project Success Criteria

The project will be considered successful if the following measurable criteria are met at the final evaluation (14 January 2027):

ID	Success criterion	How it is measured
SC-1	All 7 key features (F-1 to F-7) are implemented and all **Must** requirements in the Requirements Document pass their acceptance tests.	Acceptance test checklist in the test report
SC-2	Cached tool calls complete in under 250 ms at the 95th percentile (P95) with 20 concurrent simulated users.	Load test report
SC-3	At least 80% of student testers (minimum 5 students) find the application easy to use, and the average System Usability Scale (SUS) score is at least 70.	Usability test with SUS questionnaire
SC-4	The assistant answers at least 8 of 10 scripted compound (multi-service) questions correctly and calls the correct tools.	Scripted scenario test
SC-5	No student token, student ID, name, or e-mail address appears in any prompt sent to the LLM during the test runs.	Automated prompt-log inspection test
SC-6	A new user can start the full system with one command (`docker compose up`) on a clean machine by following the README.	Fresh-install check by a member who did not write the setup
SC-7	Every assignment is delivered on time, and the total effort stays within 70 person-hours ± 10%.	Submission dates and effort log

11. Changes Since the Concept Notes

The concept was refined after the TA's general feedback on project concepts (keep scope achievable, verify external dependencies early, consider the user experience):

Concept Notes	This document	Reason
Redis Cluster mode	Single Redis instance	Cluster adds operational work without changing the demonstrated behavior; moved out of scope (R-6)
Student SSO / OAuth2	Local mock SSO issuer with the same JWT claims	Real SSO access is not available (Section 4.3)
TLS/mTLS between all services	TLS on the public endpoint; mTLS as optional extension	Keeps the core scope achievable
Several possible LLM providers	Gemini by default behind a provider adapter	One confirmed provider; adapter keeps the option to switch
Connectors to live campus systems	Mock mode by default; live mode only for confirmed sources	Distinguishes confirmed integrations from planned ones
Error handling mainly on the server	Explicit user-facing handling of outages, missing data, slow responses, and lost connections	User experience feedback

12. Document-Specific Task Matrix

Task in creating this document	Responsible	Support / review	Effort (h)
Project name, summary, objectives	Arda Çelik	—	0.5
Scope (in/out), dependency access status	Arda Çelik	Murat Can Işık (technical check)	0.5
Target audience, key features, deliverables	Arda Çelik	—	0.5
Budget and resources, risks and mitigation	Arda Çelik	Yunus Emre Biçer (consistency with plan)	0.5
Success criteria, changes since Concept Notes	Arda Çelik	—	0.5
Guideline check (5+ features, 3+ success criteria), table of contents, author list, PDF conversion	Ahmet Sefa Efendioğlu	—	0.5
Total			**3.0**

The 2.5 hours of Arda Çelik and the 0.5 hours of Ahmet Sefa Efendioğlu are part of their Analysis & Plan phase effort in the Project Plan Document.

13. Revision History

Version	Date	Author	Change
1.0	1 Nov 2026	Arda Çelik; reviewed by Ahmet Sefa Efendioğlu	Final version for Project Assignment 1